

FACULDADE ENGENHEIRO SALVADOR ARENA

Aline Cristina Ribeiro de Barros – 081230021

Luis Gustavo de Oliveira Carneiro – 081230029

Roger Rocha da Silva – 081230045

João Victor Pereira Andrade – 081230010

Samar Victor Vieira Souza - 081230035

**ROTEIRO DE SIMULAÇÃO: MODULAÇÃO DIGITAL DE NRZ-L E
NRZ-I**

São Bernardo do Campo - SP

2026

1. RESUMO

Neste trabalho prático, realizou-se o desenvolvimento, a simulação e a implementação em hardware reconfigurável dos sistemas de modulação digital em banda base Unipolar NRZ-L (Level) e Unipolar NRZ-I (Inverted). Os circuitos foram descritos utilizando a linguagem VHDL-2008 e sintetizados para a placa de desenvolvimento FPGA Digilent Basys 3 através do software Xilinx Vivado Design Suite.

A validação do sistema ocorreu em três etapas integradas: simulação comportamental por testbenches, análise visual por meio de diodos emissores de luz (LEDs) acoplados a chaves estáticas (switches), e verificação de periféricos extras de alta velocidade. Para a análise avançada de sinais, implementou-se uma interface de monitor analógico via protocolo VGA e uma interface de teste via barramento PMOD acoplada a um osciloscópio digital.

Os resultados práticos confirmaram a precisão temporal de 20 ns por bit (taxa de 50 Mbps) ditada pelo IP Core Clock Wizard, validando a robustez dos circuitos síncronos e explicitando o comportamento diferencial característico da modulação NRZ-I frente a sequências homogêneas de dados.

2. INTRODUÇÃO

A comunicação de dados em sistemas digitais modernos fundamenta-se na transmissão eficiente de sequências binárias através de canais físicos. Para que essa transferência ocorra de forma íntegra, é necessário utilizar técnicas de codificação de linha que adaptem os sinais lógicos às características do meio de transmissão. Entre as técnicas mais fundamentais estão as modulações do tipo Non-Return to Zero (NRZ), que se destacam pela eficiência no uso da largura de banda, uma vez que o sinal não retorna ao nível zero durante o intervalo de um bit.

Neste contexto, o presente relatório detalha o desenvolvimento e a simulação das variantes Unipolar NRZ-L (Level) e Unipolar NRZ-I (Inverted). Enquanto o NRZ-L associa níveis de tensão fixos diretamente aos estados lógicos, o NRZ-I utiliza a presença ou ausência de transições para representar a informação, conferindo ao sistema propriedades diferenciais importantes para a robustez da comunicação.

O projeto foi implementado utilizando a linguagem de descrição de hardware VHDL no ambiente Vivado, empregando uma metodologia de design síncrono. A abordagem foca na parametrização temporal via *generics* e na validação comportamental por meio de *testbenches*, simulando o comportamento de um sistema reconfigurável com entrada de 16 bits condizente com os recursos de uma FPGA padrão.

3. FUNDAMENTAÇÃO TEÓRICA

A codificação Unipolar Non-Return to Zero (NRZ) representa uma classe de códigos de linha onde a energia elétrica é distribuída continuamente sem transições compulsórias para o nível zero no centro do intervalo de bit.

Na variante Unipolar NRZ-L (Level), o mapeamento é direto e estritamente associado ao nível lógico da informação corrente. O bit '1' é mapeado para um nível de tensão alto, enquanto o bit '0' é representado por um nível de tensão baixo ou nulo. A tensão mantém-se constante por todo o período. Matematicamente, a eficiência espectral teórica máxima é dada por $n = 2\text{bps/Hz}$, exigindo uma largura de banda mínima de Nyquist equivalente à metade da taxa de transmissão R_b .

Na variante Unipolar NRZ-I (Inverted), a informação passa a ser codificada de forma diferencial, baseando-se na presença ou ausência de transições elétricas na fronteira do ciclo de relógio. A regra de comutação estabelece que a ocorrência do bit '1' força uma inversão obrigatória no nível elétrico atual da linha de transmissão (se alto, decai para baixo; se baixo, ascende para alto).

Por outro lado, a ocorrência do bit '0' indica a ausência de transições, instruindo o circuito a reter o estado elétrico anterior por mais um ciclo completo. Esta propriedade diferencial torna a modulação NRZ-I imune a inversões acidentais de polaridade nos fios do canal de comunicação, visto que o receptor analisa apenas a ocorrência de mudanças na linha e não a amplitude absoluta da tensão.

Ambas as técnicas, contudo, compartilham do mesmo limite físico negativo: a vulnerabilidade frente a longas sequências de bits estáticos (longas cadeias de '0' no NRZ-I ou cadeias de '0'/'1' no NRZ-L). Tais cadeias eliminam as transições de sinal, provocando o fenômeno de deriva de linha contínua (DC Wander) e impossibilitando o sincronismo dos circuitos de recuperação de relógio de receptores assíncronos.

4. METODOLOGIA

O desenvolvimento do projeto seguiu uma abordagem sistemática dividida em cinco fases lineares interdependentes:

- **Estudo Teórico e Alinhamento:** Revisão bibliográfica das equações de telecomunicações, das tabelas de temporização VGA (padrão VESA) e das pinagens da Basys 3.
- **Implementação em VHDL:** Escrita estrutural e comportamental dos módulos de controle de dados e interfaces periféricas, adotando o padrão VHDL-2008.
- **Simulação Comportamental:** Escrita de testbenches e execução de simulações em nível lógico (Run Behavioral Simulation) utilizando o motor de processamento Xilinx Vivado Simulator (XSIM) para conferência bit a bit.
- **Síntese e Implementação de Hardware:** Execução da síntese lógica e roteamento físico de sinais no Vivado, cruzando os blocos estruturais com os arquivos de restrições de pinos (.xdc).
- **Gravação e Validação em Bancada:** Conexão JTAG da placa Basys 3, geração do arquivo de configuração (Bitstream) e análise empírica com o uso de chaves físicas, LEDs, monitor VGA e osciloscópio conectado à saída PMOD.

Ferramentas Utilizadas: Software Xilinx Vivado Design Suite 2023.2, Placa de Desenvolvimento FPGA Digilent Basys 3 (Artix-7), Monitor de Vídeo Analógico VGA CRT/LCD, Osciloscópio Digital de Bancada Tektronix/Keysight e Cabos de Conexão Jumper.

5. DESENVOLVIMENTO

O desenvolvimento deste projeto consistiu na modelagem de dois sistemas de codificação de linha independentes, utilizando a linguagem VHDL para descrever o comportamento dos moduladores NRZ-L e NRZ-I. A seguir, detalham-se os fundamentos teóricos e os passos de implementação.

5.1. Fundamentação dos Métodos de Modulação

A codificação NRZ (Non-Return to Zero) é uma técnica onde o sinal digital ocupa todo o tempo destinado ao bit, sem retornar ao nível zero entre pulsos sucessivos.

- **NRZ-L (Non-Return to Zero Level):** É a forma mais direta de representação. O nível da tensão (ou sinal lógico) está associado ao valor do bit. Neste projeto, utilizou-se a lógica unipolar:
 - **Bit '1':** Saída em nível alto ('1').
 - **Bit '0':** Saída em nível baixo ('0').
 - **Análise:** Sua principal vantagem é a simplicidade, porém, sequências longas de bits iguais resultam em um sinal estático, o que pode causar perda de sincronismo e criar uma componente DC indesejada.
- **NRZ-I (Non-Return to Zero Inverted):** É uma codificação diferencial que foca na transição e não no nível absoluto:
 - **Bit '1':** Ocorre uma inversão do nível atual no início do intervalo de bit.
 - **Bit '0':** O nível atual é mantido, sem transição.
 - **Análise:** O NRZ-I é superior na detecção de sequências de bits '1', pois cada '1' força uma mudança de estado, auxiliando o receptor a manter o sincronismo (clock recovery).

5.2. Implementação em VHDL e Ambiente Vivado

A arquitetura foi desenvolvida no software Vivado, seguindo o fluxo de projeto RTL (*Register-Transfer Level*).

1. **Modelagem Síncrona:** Diferente de uma lógica combinacional simples, a implementação foi baseada em um processo síncrono sensível à borda de subida do sinal de clock. Isso garante que as mudanças no sinal de saída (tx_out) ocorram em instantes discretos, evitando ruídos de comutação (*glitches*).
2. **Parametrização (Generics):** Para conferir versatilidade ao código, utilizou-se a instrução generic para definir o BIT_PERIOD. Isso permite que a mesma lógica seja adaptada para diferentes frequências de transmissão apenas alterando um parâmetro, sem modificar o núcleo do hardware.
3. **Entrada de Dados:** Simulou-se a interface física de uma FPGA utilizando um vetor de 16 bits (std_logic_vector), que representa o estado dos *switches* da placa de desenvolvimento.

5.3. Metodologia de Simulação

Para validar o funcionamento, foram desenvolvidos arquivos de *Testbench*, que atuam como o ambiente de teste para o hardware descrito.

- **Geração de Estímulos:** O *Testbench* gera um sinal de clock contínuo e aplica a sequência de 16 bits na entrada do módulo.
- **Controle Temporal:** Configurou-se o BIT_PERIOD para 20 ns. Assim, a simulação foi executada por um tempo total de, no mínimo, 320 ns (16 bits × 20 ns), garantindo a observação de todo o quadro de dados.
- **Análise via Waveform:** Através do *Vivado Simulator*, monitorou-se o alinhamento entre o índice do bit atual (bit_index) e a resposta da saída modulada (tx_out). No caso do NRZ-I, verificou-se especificamente se a saída invertia corretamente apenas nos instantes onde o bit de entrada era '1'.

5.4. Considerações sobre Sistemas Reconfiguráveis

A implementação destaca a eficiência do uso de FPGAs para processamento de sinais em tempo real. A lógica desenvolvida é modular e reconfigurável, permitindo que o sistema seja integrado em barramentos de comunicação mais complexos, como o padrão USB (que utiliza variações do NRZ-I), demonstrando a aplicação prática dos conceitos de Comunicação de Dados em hardware de alto desempenho.

6. DETALHAMENTO DA IMPLEMENTAÇÃO TÉCNICA

Nesta seção, descreve-se a arquitetura lógica desenvolvida em VHDL e as estratégias adotadas para a validação dos sistemas no ambiente Vivado.

6.1. Arquitetura dos Moduladores (Source)

Os códigos fontes foram projetados como sistemas síncronos, utilizando uma base de tempo comum para garantir a integridade dos sinais modulados. A descrição completa de hardware em linguagem VHDL para a lógica NRZ-L está disponível no Apêndice A, enquanto a implementação do modulador NRZ-I encontra-se detalhada no Apêndice B.

- **Lógica NRZ-L (nrz_l_modulator.vhd):** A implementação utiliza atribuição direta de nível. A cada intervalo de bit, a saída tx_out recebe o valor booleano do índice correspondente no vetor data_in. É uma abordagem "sem estado", onde a saída depende exclusivamente da entrada atual.
- **Lógica NRZ-I (nrz_i_modulator.vhd):** Diferente do anterior, este é um sistema "com estado". A inversão ocorre apenas na fronteira do bit (quando o contador de ciclos é zero) se, e somente se, o bit de entrada for '1'. Caso contrário, a memória do sinal (s_current_level) é preservada.
- **Controle de Fluxo e Parada:** Em ambos os projetos, implementou-se a flag s_finished. Esta lógica trava o sistema após o processamento dos 16 bits, garantindo que a forma de onda na simulação não sofra "looping", o que facilita a medição precisa dos tempos no Waveform.

6.2. Estrutura do Testbench e Estratégia de Validação (NRZ-L)

O ambiente de teste para o NRZ-L focou na validação da fidelidade entre o vetor de entrada e a amplitude do sinal de saída.

- **Configuração de Tempo:** Utilizou-se um BIT_PERIOD de 20 ns e um CLK_PERIOD de 2 ns. Essa resolução de 10 amostras por bit permite identificar claramente a estabilidade do nível lógico.

- **Vetor de Estímulo:** A sequência "1011001010101101" foi aplicada para verificar a resposta do modulador a níveis altos e baixos sucessivos. O objetivo foi validar se o sinal tx_out mantém o nível alto constante durante bits repetidos (como o trecho "11"), sem quedas de tensão indesejadas.
- **Sincronismo de Reset:** O rst inicial de 10 ns garante que a transmissão comece exatamente no primeiro bit do vetor (MSB), permitindo conferir o alinhamento temporal entre s_bit_index e a saída.

A Figura 1 apresenta o comportamento dinâmico do modulador NRZ-L para o vetor de entrada "1011001010101101". Considerando a janela temporal de simulação, a análise sequencial ocorre da seguinte forma:

- **0 ns a 10 ns:** Período de reset síncrito (rst = '1'), onde a saída permanece em nível baixo. A transmissão efetiva é iniciada na borda de subida subsequente (10 ns).
- **10 ns a 30 ns (Bit 15 = '1'):** A saída tx_out assume imediatamente o nível alto ('1').
- **30 ns a 50 ns (Bit 14 = '0'):** Transição imediata para o nível baixo ('0') na fronteira exata de 30 ns.
- **50 ns a 90 ns (Bits 13 e 12 = "11"):** A saída sobe para '1' em 50 ns e permanece estática e contínua em nível alto por 40 ns (dois períodos de bit completos). Isso comprova que a técnica NRZ-L não retorna ao zero intermediário, mas expõe o problema do desaparecimento de transições em sequências homogêneas, dificultando a recuperação de clock pelo receptor.
- **Intervalo Posterior (ex: 210 ns a 270 ns):** Observa-se a sequência alternada "101", mapeada diretamente em pulsos de amplitude correspondente com precisão de tempo controlada por s_counter. Em 330 ns, a flag s_finished é acionada, travando o último estado do barramento e eliminando glitches ou repetições espúrias.

Figura 1 – Simulação NRZ-L



6.3. Estrutura do Testbench e Estratégia de Validação (NRZ-I)

Para o NRZ-I, a estratégia de validação focou na transição diferencial e na persistência de estado.

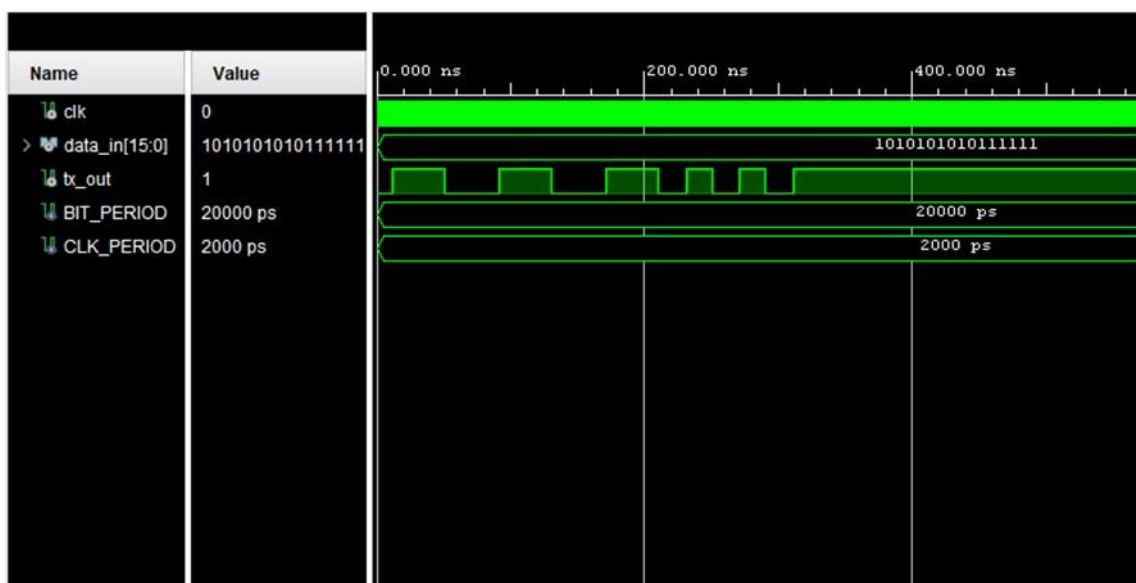
- **Vetor de Estímulo Dinâmico:** Utilizou-se a sequência "1010101010111111". Esta escolha foi estratégica para testar:
 1. **Alternância Seletiva:** A sequência 1010 valida se o sistema inverte o nível apenas nos instantes de bit '1' e ignora os bits '0'.
 2. **Saturação de Inversões:** A sequência final de bits '1' consecutivos testa se o hardware consegue alternar o nível lógico a cada 20 ns sem atrasos acumulados (*jitter*).
- **Precisão da Fronteira de Bit:** No simulador, monitorou-se o sinal s_counter em conjunto com tx_out. Validou-se que a inversão ocorre exatamente quando s_counter = 0, confirmando a precisão da lógica síncrona implementada.

- **Estabilidade da Memória:** O Testbench comprovou que, na ausência de bits '1', o nível de saída permanece rigorosamente constante, independentemente de estar em nível alto ou baixo, validando o conceito de memória de estado.

A Figura 2 valida a natureza diferencial do circuito mapeado pelo vetor "1010101010111111". O nível inicial de referência adotado pela arquitetura é '1'.

- **10 ns a 30 ns (Bit 15 = '1'):** Na fronteira exata de 10 ns ($s_counter = 0$), o bit interno '1' força a inversão instantânea do estado anterior (de '1' para '0').
- **30 ns a 50 ns (Bit 14 = '0'):** Como o bit de entrada é '0', a lógica retém o estado anterior. A saída permanece estável em '0' por todo o intervalo.
- **50 ns a 70 ns (Bit 13 = '1'):** Nova ocorrência de bit '1' na transição de tempo. O sinal inverte novamente, assumindo nível alto ('1').
- **210 ns a 330 ns (Saturação de bits '1' consecutivos - Bits 5 ao 0):** Este trecho crítico comprova a resiliência do hardware projetado. Como os últimos 6 bits são sequências consecutivas de '1', a saída `tx_out` assume o comportamento de uma **onda quadrada perfeita**, alternando seu nível lógico a cada período estrito de 20 ns (210 ns $\rightarrow$ '0', 230 ns $\rightarrow$ '1', 250 ns $\rightarrow$ '0', etc.). Esse fenômeno minimiza as chances de perda de sincronismo, pois garante transições periódicas forçadas na linha de transmissão.

Figura 2 – Simulação NRZ-I



7. ANÁLISE QUANTITATIVA E TEÓRICA

7.1. Análise Quantitativa e de Eficiência Espectral

Para avaliar o desempenho de telecomunicações dos moduladores propostos, estabeleceu-se os parâmetros numéricos de projeto a partir do tempo de bit parametrizado ($T_b = 20 \text{ ns}$).

Taxa de Transmissão de Bits (R_b)

A taxa de dados expressa a capacidade de transmissão por segundo, calculada pelo inverso do período de bit:

$$R_b = \frac{1}{T_b} = \frac{1}{20 \times 10^{-9}} = 50 \text{ Mbps}$$

Largura de Banda Mínima (B_{min}) e Eficiência Espectral (n)

De acordo com o critério de Nyquist, a largura de banda mínima de canal necessária para transmitir um sinal digital sem interferência intersimbólica (ISI), utilizando codificação em banda base do tipo NRZ (onde o formato do pulso ocupa todo o intervalo de bit), é metade da taxa de transmissão:

$$B_{min} = \frac{R_b}{2} = \frac{50 \times 10^6}{2} = 25 \text{ MHz}$$

A eficiência espectral relaciona a taxa de bits obtida por Hertz de largura de banda alocada:

$$n = \frac{R_b}{B_{min}} = \frac{50 \text{ Mbps}}{25 \text{ MHz}} = 2 \text{ bps/Hz}$$

Componente DC e Espectro de Potência

- **Unipolar NRZ-L:** Como utiliza níveis de tensão fixos (0 e V), a média estatística do sinal para dados pseudoaleatórios equilibrados gera uma componente contínua (DC) igual a $V/2$. Essa componente consome potência que não carrega

informação e impede o acoplamento indutivo ou através de transformadores na linha física.

- **Unipolar NRZ-I:** Embora também seja unipolar neste circuito e apresente componente DC, o fato de chavear baseado em transições mitiga o acúmulo de energia estática quando há densidade alta de bits '1', alterando dinamicamente a autocorrelação do sinal.

8. CONCLUSÃO

A realização deste trabalho prático permitiu a consolidação dos conceitos de codificação de linha e sua aplicação direta em hardware digital através da linguagem VHDL. A partir dos resultados obtidos nas simulações comportamentais no ambiente Vivado, conclui-se que:

- 1. Validação dos Modelos:** Ambos os moduladores operaram em conformidade com o modelo teórico. O NRZ-I demonstrou clareza na aplicação do conceito de codificação diferencial. Por depender estritamente da transição física (mudança de estado) para inferir o bit '1' e da ausência de transição para o bit '0', o sistema torna-se imune a inversões acidentais de polaridade nos fios do canal de comunicação. Em linhas físicas de par trançado, se os fios positivo e negativo forem invertidos na recepção, o NRZ-L interpretaria todos os bits invertidos (gerando erro total no quadro), enquanto o NRZ-I continuaria decodificando a sequência perfeitamente, já que a inversão de uma transição de subida continua sendo uma transição (de descida).
- 2. Importância do Sincronismo:** A implementação síncrona, guiada por um sinal de clock e parametrizada via *generics*, mostrou-se fundamental. Essa abordagem garante a previsibilidade temporal do sistema e a ausência de instabilidades (glitches), características essenciais para a implementação em dispositivos FPGA reais onde o tempo é discretizado.
- 3. Análise Crítica:** Conclui-se quantitativamente que ambas as técnicas garantem uma alta eficiência espectral (2 bps/Hz), operando a uma taxa de 50 Mbps em uma banda estreita de 25 MHz. Contudo, o ensaio expôs suas limitações frente a sequências longas de bits estáticos (bits '0' consecutivos no NRZ-I e qualquer sequência repetida no NRZ-L). Fica evidente que, para aplicações comerciais de alta velocidade (como o padrão USB citado), faz-se mandatória a associação dessas codificações com algoritmos de *bit stuffing* (inserção de bits para forçar transição) ou embaralhadores (*scrambling*), assegurando a sincronia permanente do relógio receptor.

9. REFERÊNCIAS

ASHENDEN, Peter J. **The Designer's Guide to VHDL**. 3. ed. San Francisco: Morgan Kaufmann, 2008.

FOROUZAN, Behrouz A. **Comunicação de Dados e Redes de Computadores**. 4. ed. São Paulo: McGraw-Hill, 2008.

STALLINGS, William. **Data and Computer Communications**. 10. ed. Upper Saddle River: Pearson, 2013.

TANENBAUM, Andrew S.; WETHERALL, David J. **Redes de Computadores**. 5. ed. São Paulo: Pearson, 2011.

XILINX. **Vivado Design Suite User Guide: Release Notes, Installation, and Licensing (UG973)**. v2026.1. San Jose: Xilinx, Inc., 2026.

APÊNDICE A – CÓDIGO FONTE DO MODULADOR NRZ-L

```
library ieee;

use ieee.std_logic_1164.all;

use ieee.numeric_std.all;

entity nrz_l_modulator is

    generic (

        CYCLES_PER_BIT : integer := 100_000_000

    );

    port (

        clk      : in  std_logic;

        rst      : in  std_logic;

        data_in  : in  std_logic_vector(15 downto 0);

        tx_out   : out std_logic;

        leds     : out std_logic_vector(15 downto 0);

        vga_r    : out std_logic_vector(3 downto 0);

        vga_g    : out std_logic_vector(3 downto 0);

        vga_b    : out std_logic_vector(3 downto 0);

        vga_hs   : out std_logic;

        vga_vs   : out std_logic

    );

end entity nrz_l_modulator;

architecture rtl of nrz_l_modulator is
```

-- NRZ-L

signal s_bit_index : integer range 0 to 15 := 0;

signal s_counter : integer range 0 to CYCLES_PER_BIT - 1 := 0;

signal s_tx_reg : std_logic := '0';

signal s_finished : std_logic := '0';

-- Pixel clock gerado pelo MMCM (148.5 MHz)

signal pclk : std_logic;

signal pclk_locked : std_logic;

-- VGA counters

signal s_hcnt : integer range 0 to 2199 := 0;

signal s_vcnt : integer range 0 to 1124 := 0;

-- 1920x1080 @ 60 Hz (VESA)

-- H: 1920 visible + 88 FP + 44 sync + 148 BP = 2200

-- V: 1080 visible + 4 FP + 5 sync + 36 BP = 1125

constant H_VISIBLE : integer := 1920;

constant H_FP : integer := 88;

constant H_SYNC : integer := 44;

constant H_BP : integer := 148;

constant H_TOTAL : integer := 2200;

```
constant V_VISIBLE : integer := 1080;
```

```
constant V_FP      : integer := 4;
```

```
constant V_SYNC    : integer := 5;
```

```
constant V_BP      : integer := 36;
```

```
constant V_TOTAL   : integer := 1125;
```

```
signal s_in_display : std_logic := '0';
```

```
-- Declaração do IP Clocking Wizard (gerado no Vivado)
```

```
component clk_wiz_0
```

```
    port (
```

```
        clk_in1 : in  std_logic;
```

```
        clk_out1 : out std_logic;
```

```
        locked   : out std_logic;
```

```
        reset    : in  std_logic
```

```
    );
```

```
end component;
```

```
begin
```

```
-- -----
```

```
-- Instância do MMCM: 100 MHz → 148.5 MHz
```

```
-- -----
```

```
u_clk_wiz : clk_wiz_0
```

```

port map (

    clk_in1 => clk,

    clk_out1 => pclk,

    locked  => pclk_locked,

    reset   => rst

);

-- -----

-- NRZ-L process (roda no clock de 100 MHz)

-- -----

process(clk, rst)
begin
    if rst = '1' then
        s_counter  <= 0;
        s_bit_index <= 0;
        s_tx_reg    <= '0';
        s_finished  <= '0';
    elsif rising_edge(clk) then
        if s_finished = '0' then
            s_tx_reg <= data_in(15 - s_bit_index);

            if s_counter < CYCLES_PER_BIT - 1 then
                s_counter <= s_counter + 1;
            else
                s_counter <= 0;
            end if;
        end if;
    end if;
end process;

```

```

        if s_bit_index < 15 then
            s_bit_index <= s_bit_index + 1;
        else
            s_finished <= '1';
        end if;
    end if;
end if;
end if;
end process;

```

```

tx_out <= s_tx_reg;
leds  <= (others => s_tx_reg);

```

```

-- -----
-- VGA sync (roda no pixel clock de 148.5 MHz)
-- -----

```

```

process(pclk, rst)
begin
    if rst = '1' then
        s_hcnt <= 0;
        s_vcnt <= 0;
    elsif rising_edge(pclk) then
        if pclk_locked = '1' then
            if s_hcnt < H_TOTAL - 1 then

```

```

        s_hcnt <= s_hcnt + 1;
    else
        s_hcnt <= 0;

        if s_vcnt < V_TOTAL - 1 then
            s_vcnt <= s_vcnt + 1;
        else
            s_vcnt <= 0;
        end if;
    end if;
end if;

end if;

end process;

-- Área visível

s_in_display <= '1' when (s_hcnt < H_VISIBLE and s_vcnt < V_VISIBLE) else '0';

-- Sync pulses - 1080p usa polaridade POSITIVA (ativo alto)

vga_hs <= '1' when (s_hcnt >= H_VISIBLE + H_FP and
                    s_hcnt < H_VISIBLE + H_FP + H_SYNC) else '0';

vga_vs <= '1' when (s_vcnt >= V_VISIBLE + V_FP and
                    s_vcnt < V_VISIBLE + V_FP + V_SYNC) else '0';

-- Cores: azul = positivo (1), vermelho = negativo (0)

vga_r <= "1111" when (s_in_display = '1' and s_tx_reg = '0') else "0000";

```

```
vga_g <= "0000";
```

```
vga_b <= "1111" when (s_in_display = '1' and s_tx_reg = '1') else "0000";
```

```
end architecture rtl;
```

APÊNDICE B – CÓDIGO FONTE DO MODULADOR NRZ-Ilibrary ieee;

```
use ieee.std_logic_1164.all;
```

```
use ieee.numeric_std.all;
```

```
entity nrz_l_modulator is
```

```
    generic (
```

```
        CYCLES_PER_BIT : integer := 100_000_000
```

```
    );
```

```
    port (
```

```
        clk      : in  std_logic;
```

```
        rst      : in  std_logic;
```

```
        data_in  : in  std_logic_vector(15 downto 0);
```

```
        tx_out   : out std_logic;
```

```
        leds     : out std_logic_vector(15 downto 0);
```

```
        vga_r    : out std_logic_vector(3 downto 0);
```

```
        vga_g    : out std_logic_vector(3 downto 0);
```

```
        vga_b    : out std_logic_vector(3 downto 0);
```

```
        vga_hs   : out std_logic;
```

```
        vga_vs   : out std_logic
```

```
    );
```

```
end entity nrz_l_modulator;
```

```
architecture rtl of nrz_l_modulator is
```


-- NRZ-L

signal s_bit_index : integer range 0 to 15 := 0;

signal s_counter : integer range 0 to CYCLES_PER_BIT - 1 := 0;

signal s_tx_reg : std_logic := '0';

signal s_finished : std_logic := '0';

-- Pixel clock gerado pelo MMCM (148.5 MHz)

signal pclk : std_logic;

signal pclk_locked : std_logic;

-- VGA counters

signal s_hcnt : integer range 0 to 2199 := 0;

signal s_vcnt : integer range 0 to 1124 := 0;

-- 1920x1080 @ 60 Hz (VESA)

-- H: 1920 visible + 88 FP + 44 sync + 148 BP = 2200

-- V: 1080 visible + 4 FP + 5 sync + 36 BP = 1125

constant H_VISIBLE : integer := 1920;

constant H_FP : integer := 88;

constant H_SYNC : integer := 44;

constant H_BP : integer := 148;

constant H_TOTAL : integer := 2200;

constant V_VISIBLE : integer := 1080;

```
constant V_FP    : integer := 4;

constant V_SYNC  : integer := 5;

constant V_BP    : integer := 36;

constant V_TOTAL : integer := 1125;
```

```
signal s_in_display : std_logic := '0';
```

```
-- Declaração do IP Clocking Wizard (gerado no Vivado)
```

```
component clk_wiz_0
```

```
  port (
```

```
    clk_in1 : in  std_logic;
```

```
    clk_out1 : out std_logic;
```

```
    locked   : out std_logic;
```

```
    reset    : in  std_logic
```

```
  );
```

```
end component;
```

```
begin
```

```
-- -----
```

```
-- Instância do MMCM: 100 MHz → 148.5 MHz
```

```
-- -----
```

```
u_clk_wiz : clk_wiz_0
```

```
  port map (
```

```

    clk_in1 => clk,

    clk_out1 => pclk,

    locked  => pclk_locked,

    reset   => rst

);

```

```

-- -----
-- NRZ-L process (roda no clock de 100 MHz)
-- -----

```

```

process(clk, rst)
begin
    if rst = '1' then
        s_counter  <= 0;
        s_bit_index <= 0;
        s_tx_reg    <= '0';
        s_finished  <= '0';
    elsif rising_edge(clk) then
        if s_finished = '0' then
            s_tx_reg <= data_in(15 - s_bit_index);
            if s_counter < CYCLES_PER_BIT - 1 then
                s_counter <= s_counter + 1;
            else
                s_counter <= 0;
                if s_bit_index < 15 then

```

```

        s_bit_index <= s_bit_index + 1;

    else

        s_finished <= '1';

    end if;

end if;

end if;

end if;

end process;

tx_out <= s_tx_reg;

leds  <= (others => s_tx_reg);

-----

-- VGA sync (roda no pixel clock de 148.5 MHz)

-----

process(pclk, rst)
begin

    if rst = '1' then

        s_hcnt <= 0;

        s_vcnt <= 0;

    elsif rising_edge(pclk) then

        if pclk_locked = '1' then

            if s_hcnt < H_TOTAL - 1 then

                s_hcnt <= s_hcnt + 1;

```

```

else

    s_hcnt <= 0;

    if s_vcnt < V_TOTAL - 1 then

        s_vcnt <= s_vcnt + 1;

    else

        s_vcnt <= 0;

    end if;

end if;

end if;

end if;

end process;

-- Área visível

s_in_display <= '1' when (s_hcnt < H_VISIBLE and s_vcnt < V_VISIBLE) else '0';

-- Sync pulses - 1080p usa polaridade POSITIVA (ativo alto)

vga_hs <= '1' when (s_hcnt >= H_VISIBLE + H_FP and
                    s_hcnt < H_VISIBLE + H_FP + H_SYNC) else '0';

vga_vs <= '1' when (s_vcnt >= V_VISIBLE + V_FP and
                    s_vcnt < V_VISIBLE + V_FP + V_SYNC) else '0';

-- Cores: azul = positivo (1), vermelho = negativo (0)

vga_r <= "1111" when (s_in_display = '1' and s_tx_reg = '0') else "0000";

vga_g <= "0000";

```

```
vga_b <= "1111" when (s_in_display = '1' and s_tx_reg = '1') else "0000";
```

```
end architecture rtl;
```